Database Design Document

for

Campus Resource Management and Booking System

Prepared by

Group 14

Name	Roll No.	Email
Koyyana Ram Leela Sai	B240216CS	koyyana_b240216cs@nitc.ac.in
Sharad Chandra Munugala	B241357CS	sharad_b241357cs@nitc.ac.in
Ramidi Dheeraj Reddy	B240280CS	ramidi_b240280cs@nitc.ac.in
Gunnala Mitesh Kumar	B240187CS	gunnala_b240187cs@nitc.ac.in

Instructor: Professor Prabhu Mohandas
Course: CS2011E – Database Management Systems
Date: 24 March 2026
Institution: National Institute of Technology, Calicut

Contents

1 Purpose **3**

1.1 Document Objectives . . . 3

1.2 Intended Audience and Document Overview . . . 3

1.3 Definitions, Acronyms and Abbreviations . . . 4

2 Assumptions and Constraints **5**

2.1 Assumptions . . . 5

2.2 Constraints . . . 5

3 Database-Wide Design Decisions **7**

3.1 High-Level Architecture . . . 7

3.2 Technology Stack . . . 8

4 Sequence and Data Flow Diagrams **9**

4.1 Sequence Diagrams . . . 9

4.1.1 User Authentication (Firebase Login) . . . 9

4.1.2 Booking Creation (Requires Approval) . . . 9

4.1.3 Notification . . . 11

4.1.4 User Interaction . . . 11

4.1.5 Use Case Diagram . . . 12

4.2 Data Flow Diagrams . . . 13

4.2.1 Booking Creation . . . 13

4.2.2 Approval Workflow . . . 14

4.2.3 System Architecture . . . 14

5 System Behaviour and Security **16**

5.1 Behaviour . . . 16

5.1.1 Login or Register . . . 16

5.1.2 Student . . . 16

5.1.3 Staff / Faculty . . . 16

5.1.4 Resource Manager . . . 16

5.1.5 Department Manager . . . 17

5.1.6 System Administrator . . . 17

5.2 DBMS Platform . . . 17

5.3 Security Requirements 17

6 Models and Schema **19**

6.1 Entity-Relation Model 19

6.2 Relational Schema 19

6.3 Normalisation 20

6.3.1 First Normal Form (1NF) 20

6.3.2 Second Normal Form (2NF) 21

6.3.3 Third Normal Form (3NF) 21

6.4 Schema Description and Data Formats 22

6.4.1 USER Table 22

6.4.2 BOOKING Table 22

6.4.3 RESOURCE Table 23

6.4.4 DEPARTMENT Table 23

6.4.5 BOOKING_STATUS Table 23

6.4.6 RESOURCE_CATEGORY Table 24

6.4.7 RECURRENCE Table 24

6.4.8 PRIORITY Table 24

6.4.9 APPROVAL Table 24

6.4.10 LOG Table 25

6.4.11 NOTIFICATION Table (Weak Entity) 25

6.4.12 AUDIT_LOG Table 26

Chapter 1

Purpose

This Database Design Document (DDD) for the Campus Resource Management and Booking System (CRMBS) describes the database management system chosen based on the requirements defined in the Software Requirements Specification (SRS). It explains how the Entity-Relationship (ER) model, developed from the system's use cases, is transformed into a relational schema for PostgreSQL 15 or later. The design focuses on maintaining data consistency, ensuring data integrity, and supporting efficient and conflict-free resource booking within the system.

1.1 Document Objectives

The main objectives of this Database Design Document are:

- To describe the design and structure of the CRMBS database along with its key components and how they interact with the system through the DBMS.
- To provide a clear foundation for implementing the database and supporting software modules, helping developers understand how the application will be built.
- To explain the mechanisms used to prevent booking conflicts, manage concurrent requests, and handle priority-based scheduling.
- To document important system features such as audit logging, approval workflow, and notifications to ensure transparency and proper system monitoring.

1.2 Intended Audience and Document Overview

Audience for this Document

This document is mainly intended for the course instructor, Professor Prabhu Mohandas, for project evaluation. It is also useful for software developers, database administrators, and testers involved in the development and deployment of the system. The document explains the structure and design of the CRMBS database so that stakeholders can clearly understand how the system works and how the database supports its functionality.

Document Overview

- **Section 2 (Assumptions and Constraints):** Describes the key assumptions and limitations considered during system development.
- **Section 3 (Database-Wide Design Decisions):** Explains the overall database architecture, technology stack, and major design choices.
- **Section 4 (Sequence and Data Flow Diagrams):** Shows how data moves through important system processes.
- **Section 5 (System Behaviour and Security):** Describes user interactions, access control, and security mechanisms.
- **Section 6 (Models and Schema):** Presents the ER model, relational schema, normalisation, and detailed database structure.

1.3 Definitions, Acronyms and Abbreviations

S.No	Abbreviation / Term	Definition
1	HTML	Hyper Text Markup Language
2	SRS	Software Requirements Specifications
3	NITC	National Institute of Technology, Calicut
4	UMS	University Management System
5	User	A student or staff of the University
6	Admin	Administrator
7	Database	Stores the catalogue of resources available and details of the users
8	DBMS	Database Management System
9	1NF	First Normal Form
10	2NF	Second Normal Form
11	3NF	Third Normal Form

Chapter 2

Assumptions and Constraints

2.1 Assumptions

The following assumptions were considered while designing and developing the CRMBS database:

- **User Authentication and Authorisation:** The system supports multiple user roles such as System Administrator, Resource Manager, Department Manager, Staff/Faculty, and Student, with role-based access control enforced in the database.
- **Security and Data Protection:** User authentication and credential management are handled through Firebase Authentication, and sensitive credential data is not stored in the PostgreSQL database.
- **Data Integrity and Backup:** PostgreSQL provides mechanisms for data recovery and backup, and regular backups are expected to be maintained to prevent data loss.
- **Third-Party Dependencies:** The system relies on Firebase Authentication for user identity management and an email service for sending notifications, which must be properly configured during deployment.
- **Network and Browser Support:** Users are expected to access the system through modern web browsers with a stable internet connection to ensure smooth operation.
- **System Infrastructure:** The deployment environment should have sufficient resources to handle multiple users and booking requests efficiently.

These assumptions help ensure that the database and system operate reliably and support the intended functionality of the CRMBS.

2.2 Constraints

The following constraints are considered in the CRMBS system:

- Booking conflicts are controlled at the PostgreSQL database level using the `create_booking` procedure to prevent overlapping bookings.
- User login and authentication are handled using Firebase, and no passwords are

stored in the PostgreSQL database.

- Only System Administrators or HODs can assign or change user roles, and users cannot change their own roles.
- The audit log keeps a record of system activities and cannot be edited or deleted.
- A user must have a valid role in the database to access system features.
- The system should be able to handle many booking requests at the same time without slowing down.
- Advance booking limits for resources are controlled through the database.

These constraints ensure that the system remains secure and bookings are properly managed.

Chapter 3

Database-Wide Design Decisions

3.1 High-Level Architecture

The CRMBS system follows a simple 3-tier architecture with three main layers:

1. Presentation Layer (Frontend – React 18)

- Provides the user interface such as dashboard, booking calendar, and resource list.
- Users can view resources, check availability, and make bookings through the web application.
- The frontend communicates with the backend using REST APIs over HTTPS.
- A step-by-step booking process helps users select a resource, check availability, and confirm the booking.

2. Application Layer (Backend – Node.js or FastAPI)

- Handles user requests and processes system logic.
- Verifies user login using Firebase authentication.
- Checks user roles and permissions before allowing actions like booking or approval.
- Sends booking requests to the database and manages notifications such as emails.

3. Data Layer (PostgreSQL Database)

- Stores all system data including users, resources, bookings, approvals, notifications, and audit logs.
- Prevents booking conflicts and ensures data consistency.
- Maintains records of system activities for monitoring and tracking changes.

This three-layer architecture helps keep the system organised, secure, and easy to maintain.

3.2 Technology Stack

Component	**Technology**
Frontend	React , Tailwind css
Backend	Node.js with Express
Database	PostgreSQL
Authentication	JWT (via Firebase)

Table 3.1: Technology Stack

Chapter 4

Sequence and Data Flow Diagrams

4.1 Sequence Diagrams

4.1.1 User Authentication (Firebase Login)

The sequence diagram below illustrates the complete authentication flow from credential entry through Firebase JWT verification to role-based dashboard redirection.

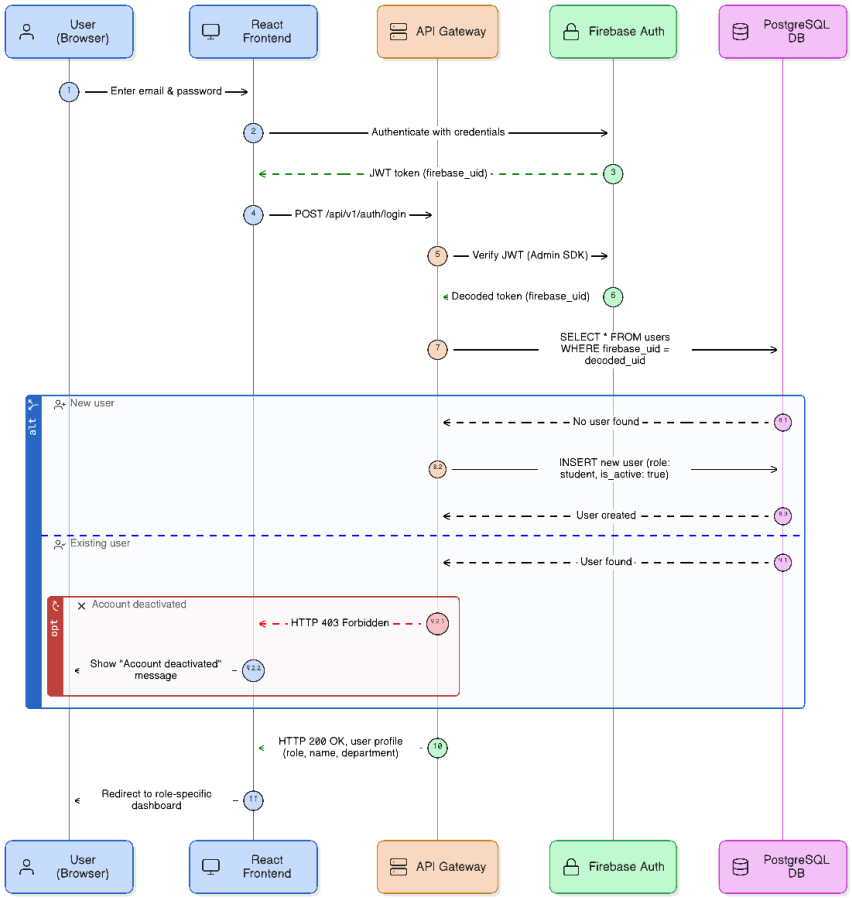

Figure 4.1: User Authentication (Firebase Login)

4.1.2 Booking Creation (Requires Approval)

The sequence diagram below shows the end-to-end booking creation process, including JWT verification, conflict detection via the `create_booking` procedure, conditional approval routing, and notification dispatch.

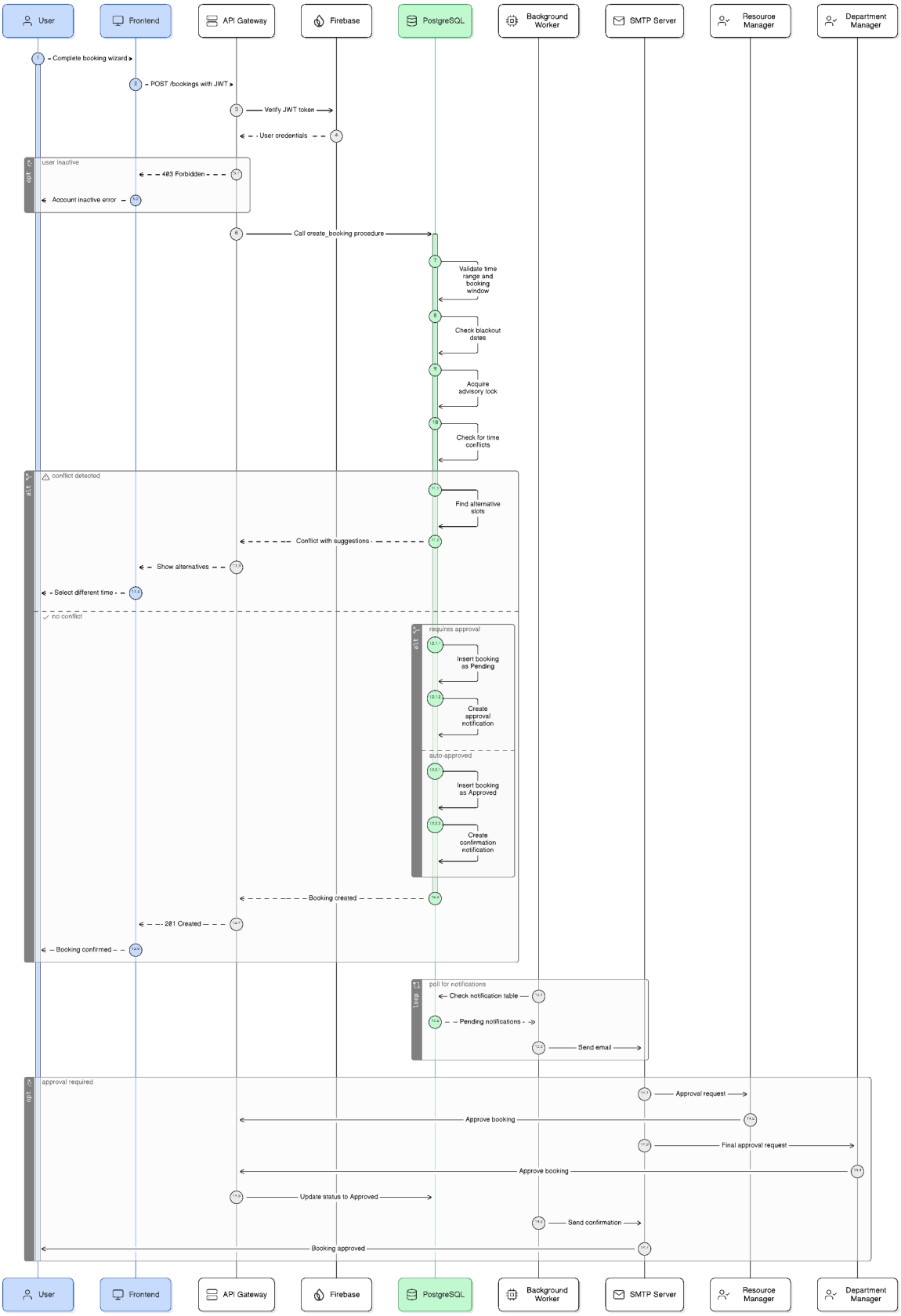

Figure 4.2: Booking Creation (Requires Approval)

4.1.3 Notification

The sequence diagram below depicts the background notification worker loop, which polls the PostgreSQL database every 60 seconds for pending notifications and dispatches emails via the SMTP server, with success/failure handling and daily reminder checks.

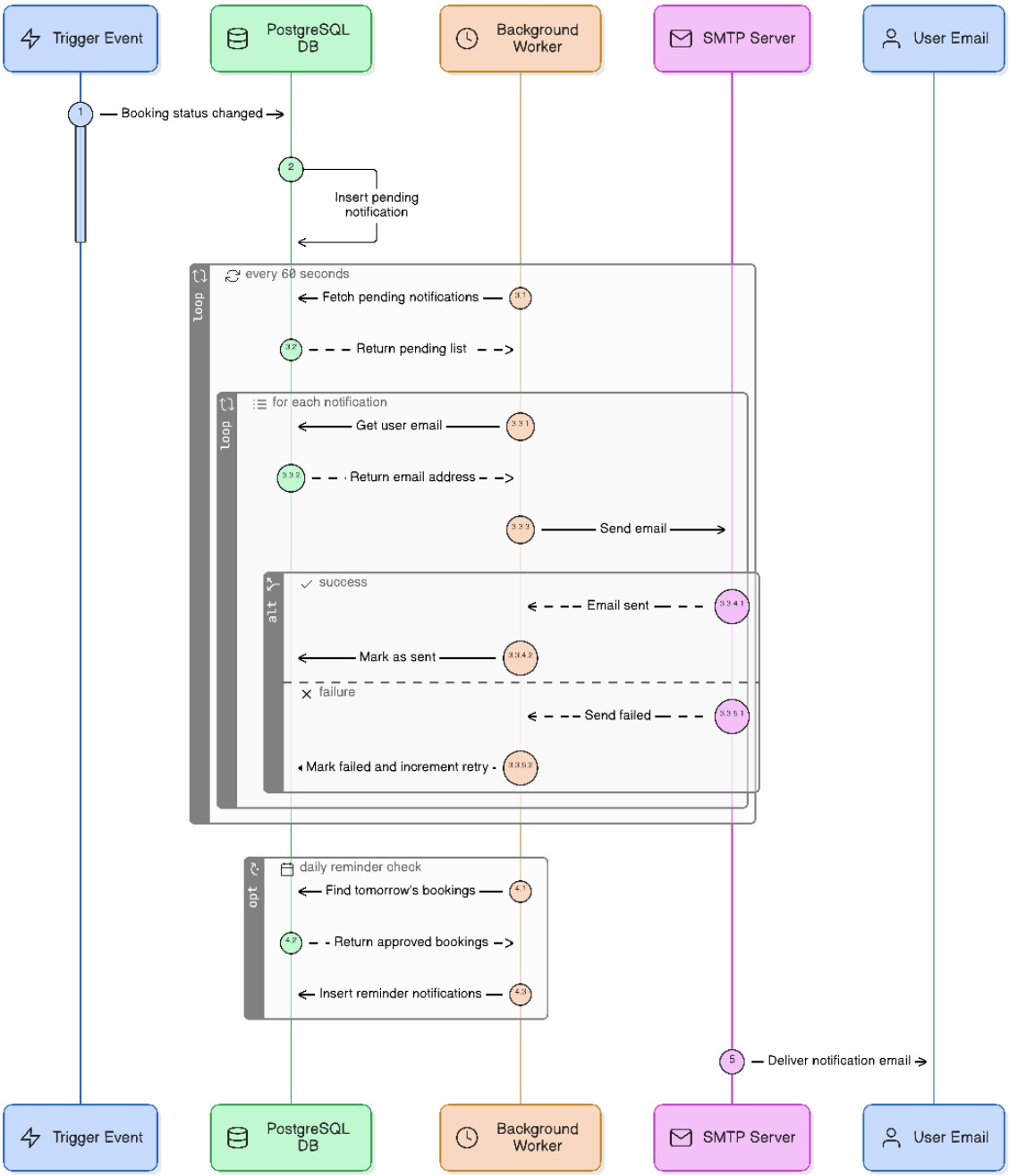

Figure 4.3: Notification

4.1.4 User Interaction

The sequence diagram below shows the resource search and availability flow, where the frontend polls the API gateway every 30 seconds, JWT is validated, indexed resource

queries are executed, and time-slot pickers are presented to the user.

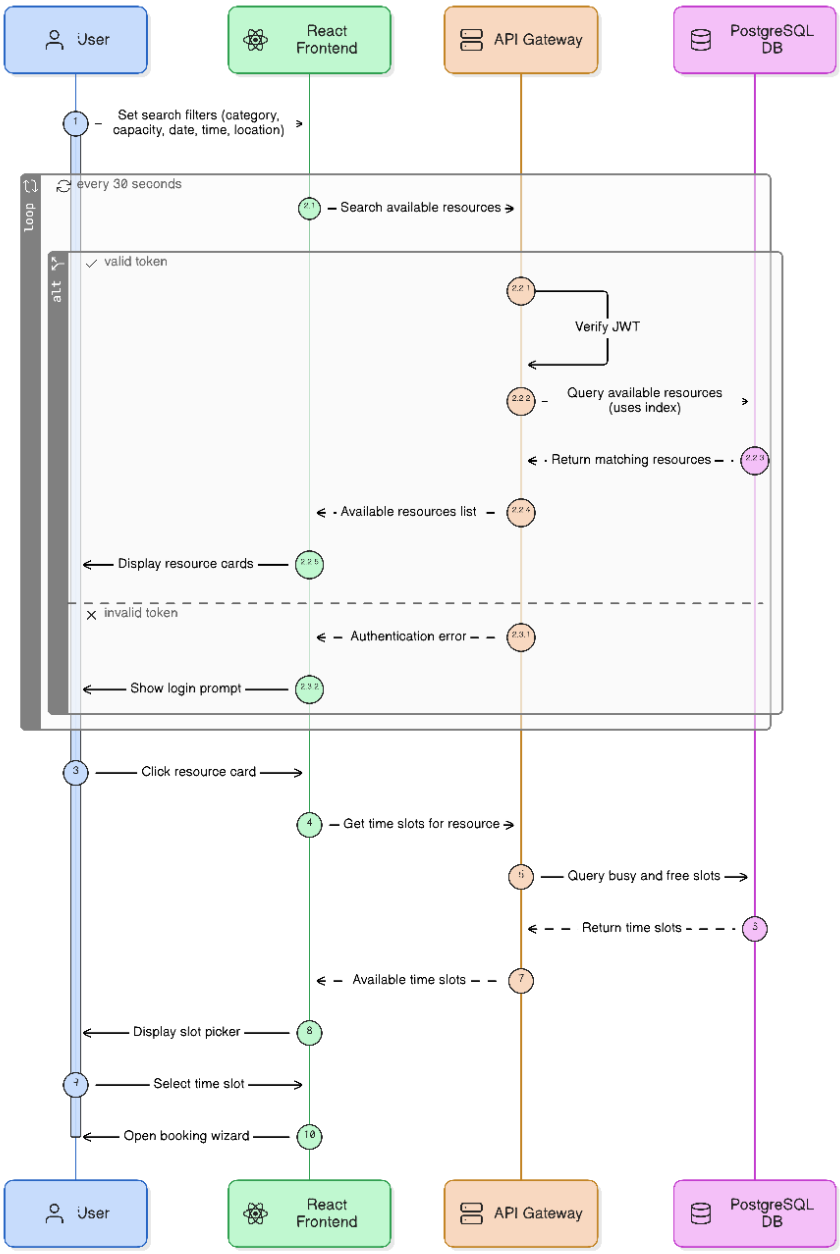

Figure 4.4: User Interaction

4.1.5 Use Case Diagram

The use case diagram below captures the full set of actor interactions across System Administration, Common Actions, Resource Management, and Department Management subsystems.

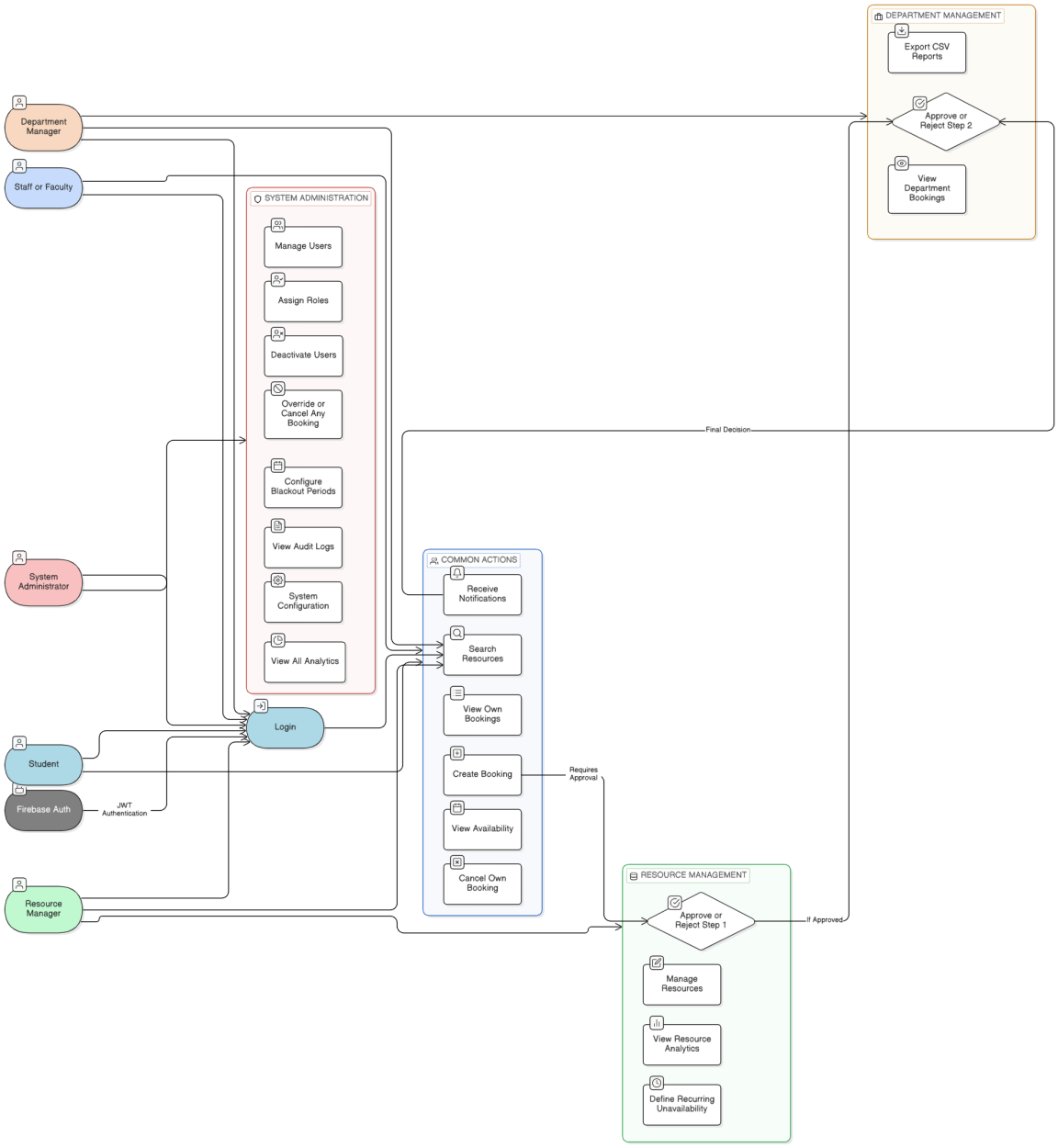

Figure 4.5: Use Case Diagram

4.2 Data Flow Diagrams

4.2.1 Booking Creation

The data flow diagram below illustrates the state transitions of a booking from initiation through priority recording, conflict detection, and final states (Pending, Approved, Rejected, Completed, Cancelled), with audit-log and notification triggers at each stage.

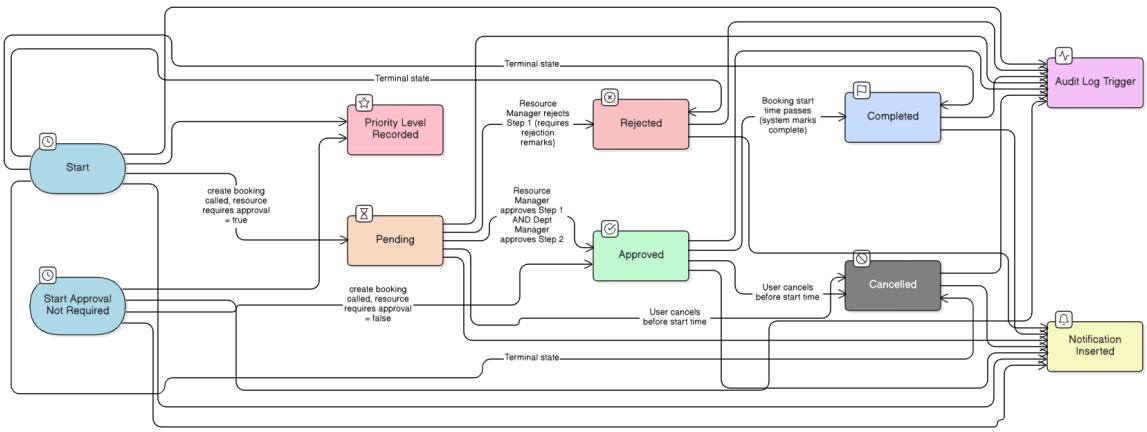

Figure 4.6: Booking Creation

4.2.2 Approval Workflow

The swimlane diagram below shows the two-step approval workflow across the User, System, Resource Manager, and Department Manager lanes, including auto-rejection on deadline timeout and audit logging at each decision point.

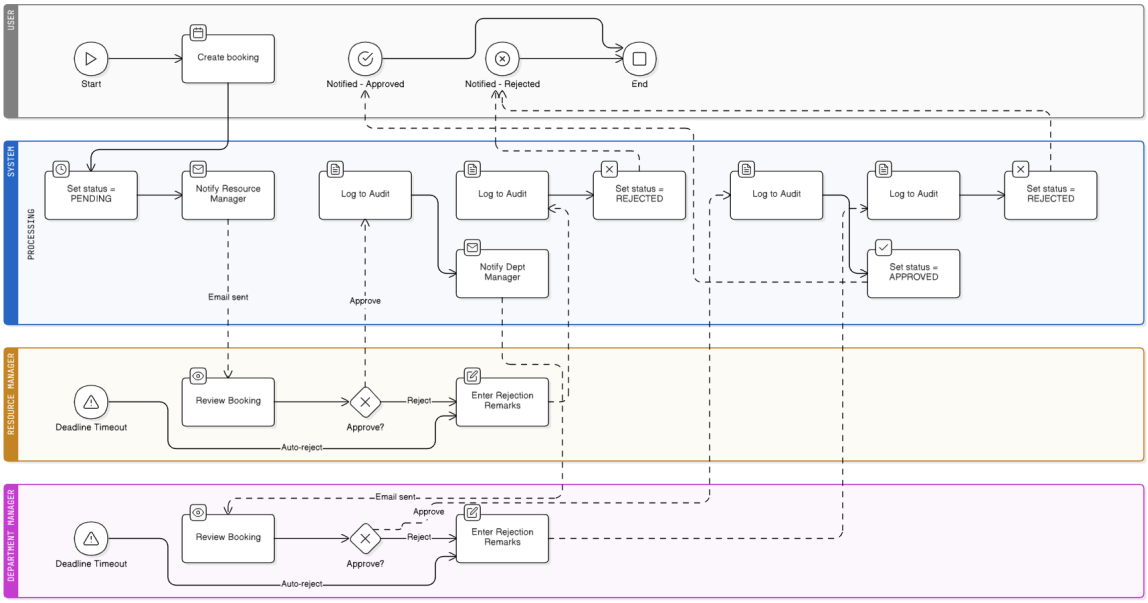

Figure 4.7: Approval Workflow

4.2.3 System Architecture

The architecture diagram below provides a holistic view of all system layers: Client Layer (React SPA, Mobile Browser, REST API Consumers), API Gateway (Node.js, Firebase JWT Middleware, RBAC, Rate Limiter), Background Workers, Application Services, and the PostgreSQL database subdivided into Operational Tables, DB Logic, Core Tables, and Lookup Tables.

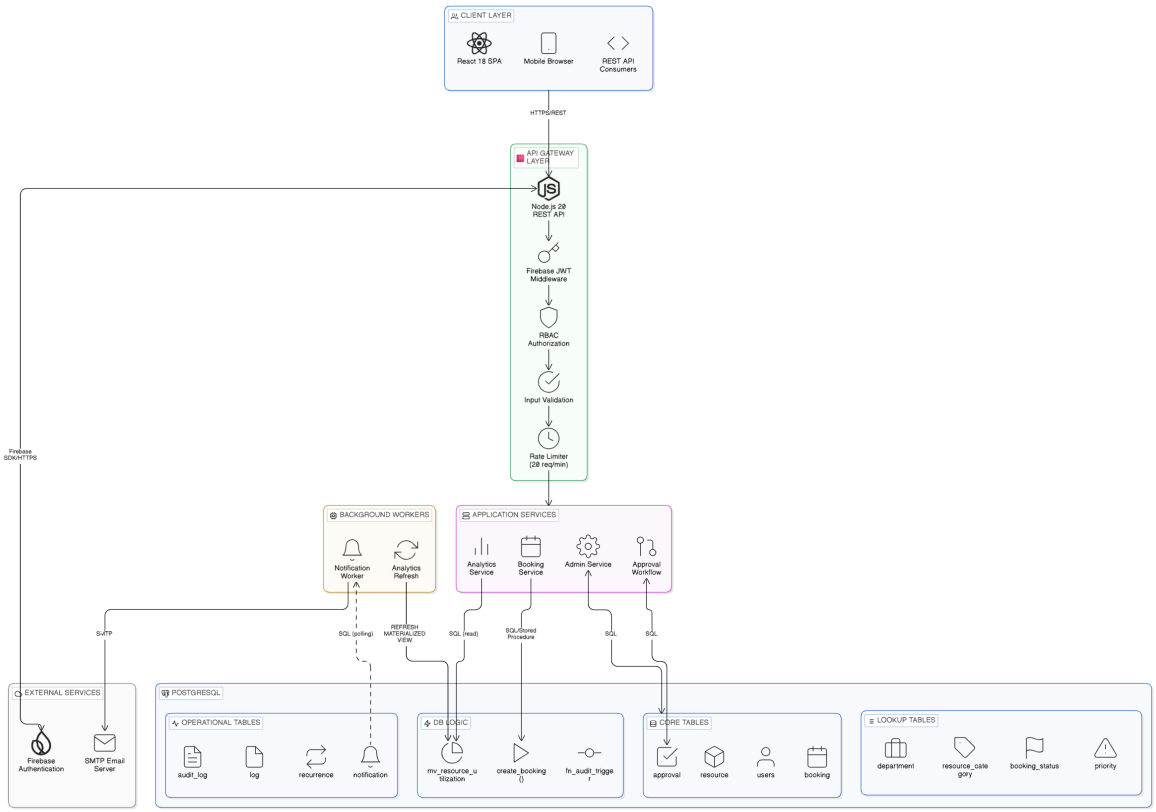

Figure 4.8: System Architecture Diagram

Chapter 5

System Behaviour and Security

5.1 Behaviour

5.1.1 Login or Register

Users open the application and log in using their institutional email and password through Firebase Authentication. After the first login, a user record is created with the default role of Student. Only System Administrators or HODs can change user roles. Users can also reset their password using Firebase.

5.1.2 Student

Students can:

- Search and view available campus resources.
- Book resources by selecting date and time.
- View and cancel their own bookings before the start time.
- Receive notifications about booking status.

Students have the lowest booking priority and limited booking permissions.

5.1.3 Staff / Faculty

Staff and Faculty can:

- Perform all student booking activities.
- Book resources for academic or official work.
- View their booking history and active bookings.

They have higher priority than students.

5.1.4 Resource Manager

Resource Managers can:

- Add and manage resources in their category.
- Set unavailable time periods for resources.
- Approve booking requests for restricted resources.

- View resource usage reports.

5.1.5 Department Manager

Department Managers can:

- Approve booking requests at the department level.
- View department bookings.
- Access booking reports and analytics.

5.1.6 System Administrator

System Administrators can:

- Manage users and assign roles.
- Manage all resources and bookings.
- Cancel or modify bookings if needed.
- Set system blackout periods.
- Access full system controls and reports.

5.2 DBMS Platform

The CRMBS is a web-based system that uses PostgreSQL as the main database and runs on a server environment. Users log in using Firebase Authentication and are taken to dashboards based on their roles. The system prevents booking conflicts at the database level to ensure that the same resource is not booked at the same time. All important system activities are recorded in an audit log for tracking and monitoring. The system also provides reports and analytics to view booking and resource usage. A simple 3-step booking process and calendar view are provided to make resource booking easy for all users.

5.3 Security Requirements

1. Access Control

- User roles are stored in the database and checked whenever a user performs any action.
- Only System Administrators or HODs can assign or change user roles.
- System activity records (audit log) cannot be edited or deleted.

2. Authentication and Authorisation

- User login and password management are handled by Firebase, and no passwords are stored in the database.
- The system checks user identity and role before allowing access to features.

- Users can only access features based on their assigned role.

3. Data Protection

- The system prevents invalid or harmful database queries.
- The database ensures that the same resource is not booked at the same time.
- Multiple booking requests are handled safely to avoid duplicate bookings.

4. Data Backup and Recovery

- The database supports backup and recovery to prevent data loss.
- Regular backups should be maintained to restore data if needed.

5. Monitoring and Logging

- System activities and errors are recorded for monitoring.
- Slow system operations are tracked to improve performance.

6. System Availability

- The system is designed to run continuously with minimal downtime.
- Maintenance is done during low-usage periods to avoid user impact.

Chapter 6

Models and Schema

6.1 Entity-Relation Model

The ER diagram below represents all entities and relationships in the CRMBS: User, Department, Resource, Resource_Category, Booking, Booking_Status, Approval, Priority, Recurrence, Log, Notification, and Audit_Log.

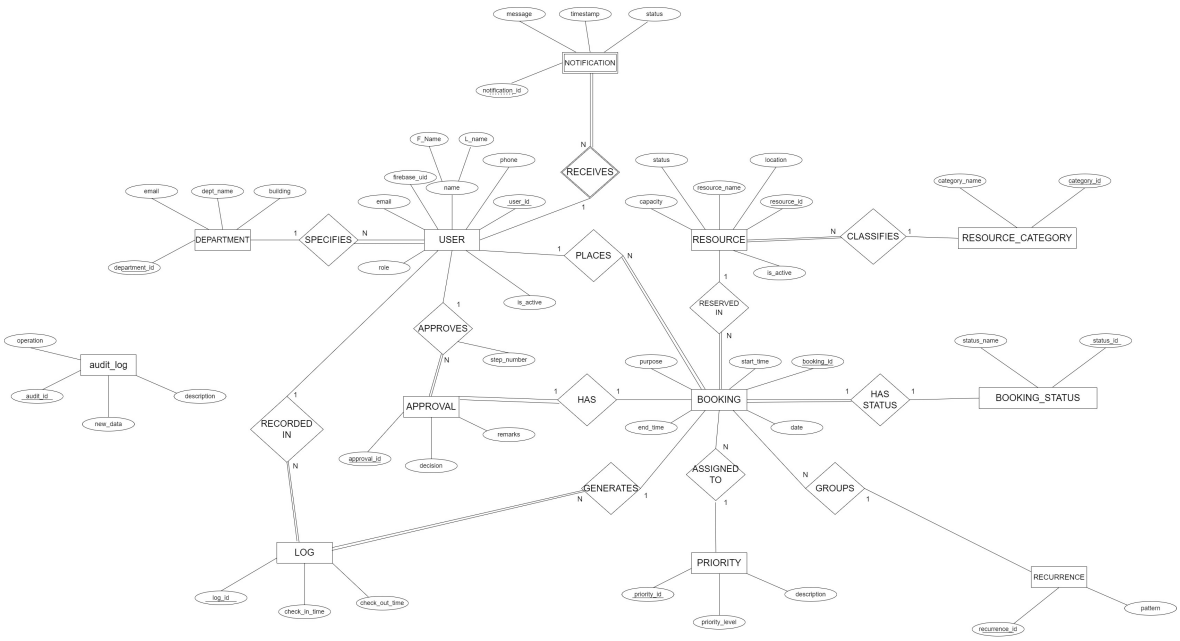

Figure 6.1: Entity-Relation (ER) Diagram

6.2 Relational Schema

The relational schema diagram below maps the ER model to tables with primary keys (underlined), foreign keys, and referential-integrity links.

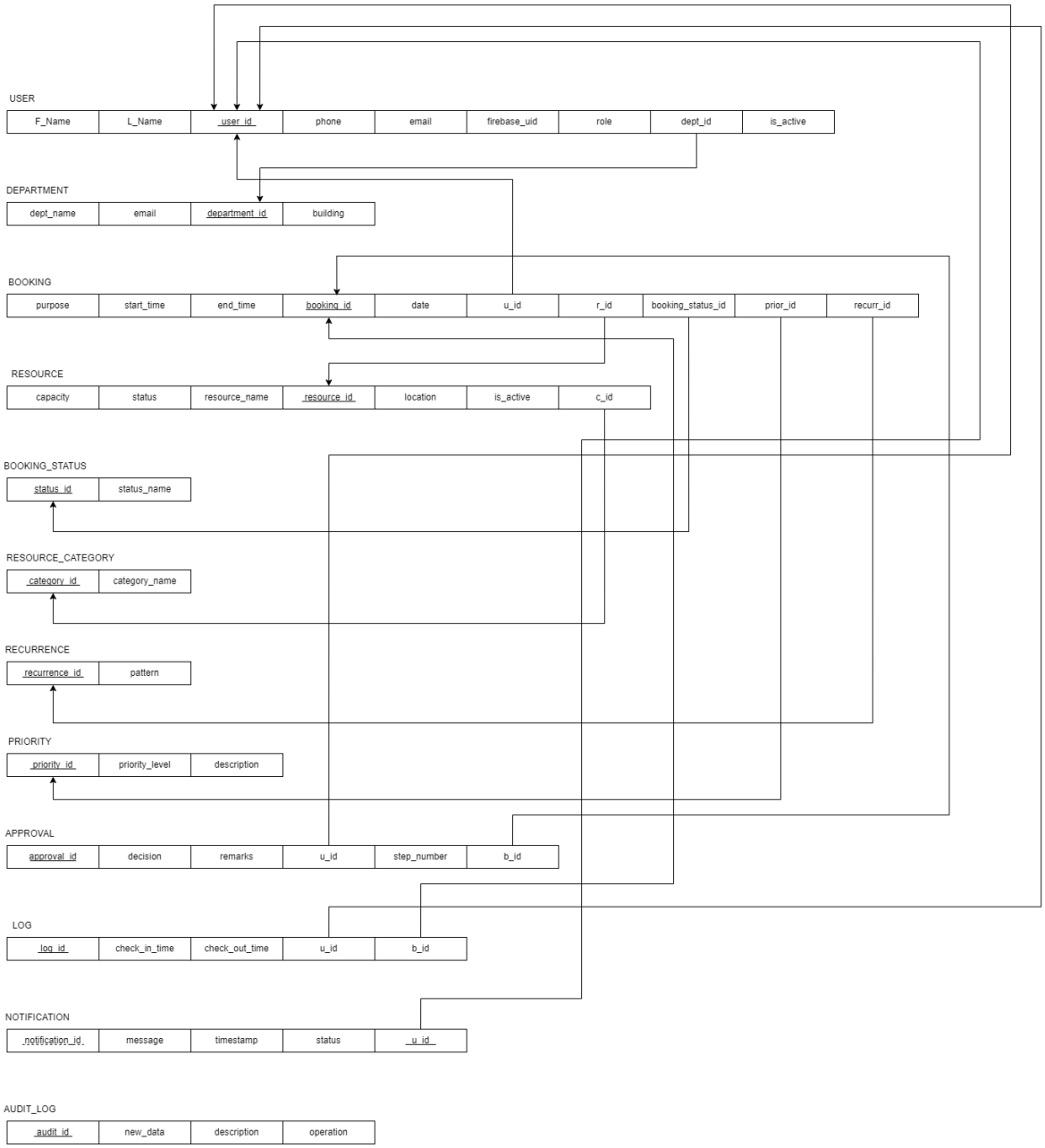

Figure 6.2: Relational Schema Diagram

6.3 Normalisation

6.3.1 First Normal Form (1NF)

- All tables have a primary key to uniquely identify each record.
- Each column contains atomic values (no repeating groups or arrays).
- There are no duplicate rows in any table.

Examples:

- In the `USER` table, each user is uniquely identified by `user_id`, and attributes like

`F_name`, `L_name`, `email`, and `phone` store atomic values.
- In the `BOOKING` table, `booking_id` uniquely identifies each booking, and attributes like `purpose`, `start_time`, and `end_time` are atomic.
- In the `NOTIFICATION` table, `notification_id` uniquely identifies each notification with atomic `message` and `status` fields.
- In the `PRIORITY` table, `priority_id` uniquely identifies priority levels with atomic attributes.

Thus, all tables satisfy the requirements of 1NF.

6.3.2 Second Normal Form (2NF)

- The database is already in 1NF.
- All non-key attributes are fully functionally dependent on the primary key.
- For tables with composite keys (e.g., `USER`, `BOOKING`, `DEPARTMENT`, and `RESOURCE`), there are no partial dependencies:
  - In the `USER` table, all attributes depend fully on `user_id`.
  - In the `DEPARTMENT` table, all attributes depend on `department_id`.
  - In the `RESOURCE` table, all attributes depend on `resource_id`.
  - In the `BOOKING` table, all attributes depend on `booking_id`.

Thus, all tables meet the criteria for 2NF.

6.3.3 Third Normal Form (3NF)

- The database is already in 2NF.
- There are no transitive dependencies in any table. In all tables, non-key attributes depend only on the primary key and not on other non-key attributes.

Examples:

- In the `USER` table, department-related details are stored separately in the `DEPARTMENT` table and referenced using `dept_id`, avoiding transitive dependency.
- In the `RESOURCE` table, category details are separated into the `RESOURCE_CATEGORY` table using `c_id`.
- In the `BOOKING` table, status, priority, and recurrence details are stored in separate tables (`BOOKING_STATUS`, `PRIORITY`, `RECURRENCE`) and referenced via foreign keys.
- In the `APPROVAL`, `LOG`, and `NOTIFICATION` tables, relationships are maintained using foreign keys (`u_id`, `b_id`) without storing redundant data.

Thus, all tables satisfy the requirements of 3NF.

6.4 Schema Description and Data Formats

6.4.1 USER Table

Description: Entity that stores user information including first name, last name, email, phone number, role, and account status. It also stores the `firebase_uid`, which links the user account to Firebase Authentication.

Attribute	Data Type	Constraints / Notes
`user_id`	`INT`	Primary Key
`name`	`VARCHAR(255)`	
`email`	`VARCHAR(255)`	Unique
`dept_id`	`INT`	FK → `DEPARTMENT(department_id)`
`phone`	`VARCHAR(15)`	
`role`	`ENUM`	`'student', 'faculty', 'admin'`
`firebase_uid`	`INT`	Unique
`is_active`	`BOOLEAN`	

Table 6.1: USER Table – Attributes and Data Formats

6.4.2 BOOKING Table

Description: Central table that stores all booking information, including booking date, start time, end time, purpose, priority level, user, resource, and booking status.

Attribute	Data Type	Constraints / Notes
`booking_id`	`INT`	Primary Key
`start_time`	`TIME`	
`end_time`	`TIME`	
`date`	`DATE`	
`purpose`	`VARCHAR(255)`	
`u_id`	`INT`	FK → `USER(user_id)`
`r_id`	`INT`	FK → `RESOURCE(resource_id)`
`booking_status_id`	`INT`	FK → `BOOKING_STATUS(status_id)`
`prior_id`	`INT`	FK → `PRIORITY(priority_id)`
`recurr_id`	`INT`	FK → `RECURRENCE(recurrence_id)`

Table 6.2: BOOKING Table – Attributes and Data Formats

6.4.3 RESOURCE Table

Description: Entity that stores details of individual resources including resource name, capacity, location, status, and `is_active`.

Attribute	Data Type	Constraints / Notes
`resource_id`	`INT`	Primary Key
`role`	`ENUM`	`'professor', 'hod'`
`capacity`	`VARCHAR(225)`	
`status`	`VARCHAR(255)`	
`resource_name`	`VARCHAR(255)`	
`location`	`VARCHAR(255)`	
`is_active`	`BOOLEAN`	
`c_id`	`INT`	FK → `RESOURCE_CATEGORY(category_id)`

Table 6.3: RESOURCE Table – Attributes and Data Formats

6.4.4 DEPARTMENT Table

Description: Entity that stores department information such as department name, building location, and contact email. This table helps organise users by department.

Attribute	Data Type	Constraints / Notes
`department_id`	`INT`	Primary Key
`email`	`VARCHAR(255)`	Unique
`dept_name`	`VARCHAR(255)`	
`building`	`VARCHAR(255)`	

Table 6.4: DEPARTMENT Table – Attributes and Data Formats

6.4.5 BOOKING_STATUS Table

Description: Entity that defines the possible booking states such as Pending, Approved, Rejected, Cancelled, and Completed.

Attribute	Data Type	Constraints
`status_id`	`INT`	Primary Key
`status_name`	`VARCHAR(255)`	

Table 6.5: BOOKING_STATUS Table – Attributes and Data Formats

6.4.6 RESOURCE_CATEGORY Table

Description: Entity that stores categories of resources such as classrooms, laboratories, meeting rooms, or equipment. It also defines booking policies such as advance booking limits.

Attribute	Data Type	Constraints
`category_id`	`INT`	Primary Key
`category_name`	`VARCHAR(255)`	

Table 6.6: RESOURCE_CATEGORY Table – Attributes and Data Formats

6.4.7 RECURRENCE Table

Description: Entity that stores information related to recurring bookings, such as weekly booking patterns, and links multiple booking occurrences together.

Attribute	Data Type	Constraints
`recurrence_id`	`INT`	Primary Key
`pattern`	`VARCHAR(255)`	

Table 6.7: RECURRENCE Table – Attributes and Data Formats

6.4.8 PRIORITY Table

Description: Entity that stores priority levels assigned to different user roles. These priority levels are used during booking scheduling and conflict resolution.

Attribute	Data Type	Constraints
`priority_id`	`INT`	Primary Key
`priority_level`	`INT`	
`description`	`TEXT`	

Table 6.8: PRIORITY Table – Attributes and Data Formats

6.4.9 APPROVAL Table

Description: Entity that stores approval workflow information. Each booking requiring approval will have approval records indicating which user approved or rejected the request and at which step.

Attribute	Data Type	Constraints / Notes
approval_id	INT	Primary Key
u_id	INT	FK → USER(user_id)
decision	VARCHAR(255)	
remarks	TEXT	
step_number	INT	
b_id	INT	FK → BOOKING(booking_id)

Table 6.9: APPROVAL Table – Attributes and Data Formats

6.4.10 LOG Table

Description: Entity that stores application-level activity logs that record important system events for user-visible history.

Attribute	Data Type	Constraints / Notes
log_id	INT	Primary Key
check_in_time	TIME	
check_out_time	TIME	
b_id	INT	FK → BOOKING(booking_id)
u_id	INT	FK → USER(user_id)

Table 6.10: LOG Table – Attributes and Data Formats

6.4.11 NOTIFICATION Table (Weak Entity)

Description: Entity that stores notification messages that are sent to users.

Attribute	Data Type	Constraints / Notes
notification_id	INT	Partial Attribute
u_id	INT	FK → USER(user_id)
timestamp	TIMESTAMP	
message	VARCHAR(255)	
status	ENUM	'SENT', 'NOT SENT' (Default: 'NOT SENT')

Table 6.11: NOTIFICATION Table – Attributes and Data Formats

6.4.12 AUDIT_LOG Table

Description: Entity that stores a permanent history of all database changes. This table captures insert, update, and delete operations using database triggers.

Attribute	Data Type	Constraints
`audit_id`	`INT`	Primary Key
`new_data`	`VARCHAR(255)`	
`operation`	`VARCHAR(255)`	
`description`	`TEXT`	

Table 6.12: AUDIT_LOG Table – Attributes and Data Formats